

Car Rental Management System

Project Report

Student Name: Sarthak Shukla

Registration Number: 25BAI10023

Subject: Project Report - JAVA

Submission Date: 18-09-2026

1. Introduction

Car rental outlets frequently manage their fleet, customers, and bookings using manual registers or spreadsheets. This is error-prone and makes it hard to answer simple questions quickly, such as which cars are free right now, or how much a customer owes for a multi-day rental of a premium vehicle.

This project implements a **Car Rental Management System**: a console-based Java application built around core object-oriented programming concepts (encapsulation, inheritance, and polymorphism) that models cars, customers, and rentals, and enforces consistent business rules in code rather than relying on manual bookkeeping.

2. Problem Statement

Manual tracking of a rental fleet leads to double-booking already-rented cars, inconsistent or manually-calculated billing (particularly for premium/luxury vehicles with additional daily charges), and no reliable record of past rentals. The system must provide a single, consistent source of truth for fleet availability and rental records during each session.

Scope

- Single-location car rental operation (not multi-branch).
- Console/terminal based interface — no GUI or web frontend.
- In-memory data storage for the current session.
- Supports standard cars and luxury cars (with an added daily surcharge).
- Out of scope: persistent database storage, payment gateway integration, multi-user concurrent access, and a graphical user interface.

Target Users

- **Rental shop staff / operator** — the primary user who adds cars, processes rentals/returns, and checks history.
- **Shop owner / manager** — benefits indirectly from accurate fleet and rental visibility.

3. Functional Requirements

The system is organised into three major functional modules, as required by the project guidelines:

Module 1 — Car (Fleet) Management

- Add a new car to the fleet as either a standard car or a luxury car.
- View all cars that are currently available for rent.

Module 2 — Rental & Return Processing

- Rent an available car to a customer for a specified number of days.
- Automatically compute the total rental amount, including the luxury surcharge where applicable.
- Return a car using its Rental ID, making the car available again.

Module 3 — Customer & History Tracking

- Capture customer name and phone number at the time of rental.
- Maintain and display a complete rental history (active and returned), including customer, car, number of days, amount, and status.

Input/output structure: the system reads operator input from the console (menu choices, car details, customer details, rental/return IDs) and prints confirmations, listings, and error messages back to the console. The logical workflow of these interactions is shown in the Process Flow diagram in Section 7.

4. Non-Functional Requirements

Reliability

A car's `available` flag is checked before every rental and toggled consistently on rent/return, so the fleet state never becomes inconsistent during a session.

Usability

The console menu is numbered and redisplayed after every action; error messages (e.g. "Car not found!") are shown in plain language.

Maintainability

Each responsibility is isolated in its own class, and `LuxuryCar` extends `Car` rather than duplicating its fields, so new car types or pricing rules can be added with minimal change to existing code.

Error Handling Strategy

User input is validated before use: renting checks for a missing/unavailable car and a non-positive day count; returning checks that the Rental ID exists and is still active; invalid menu choices show a default message instead of crashing.

Performance

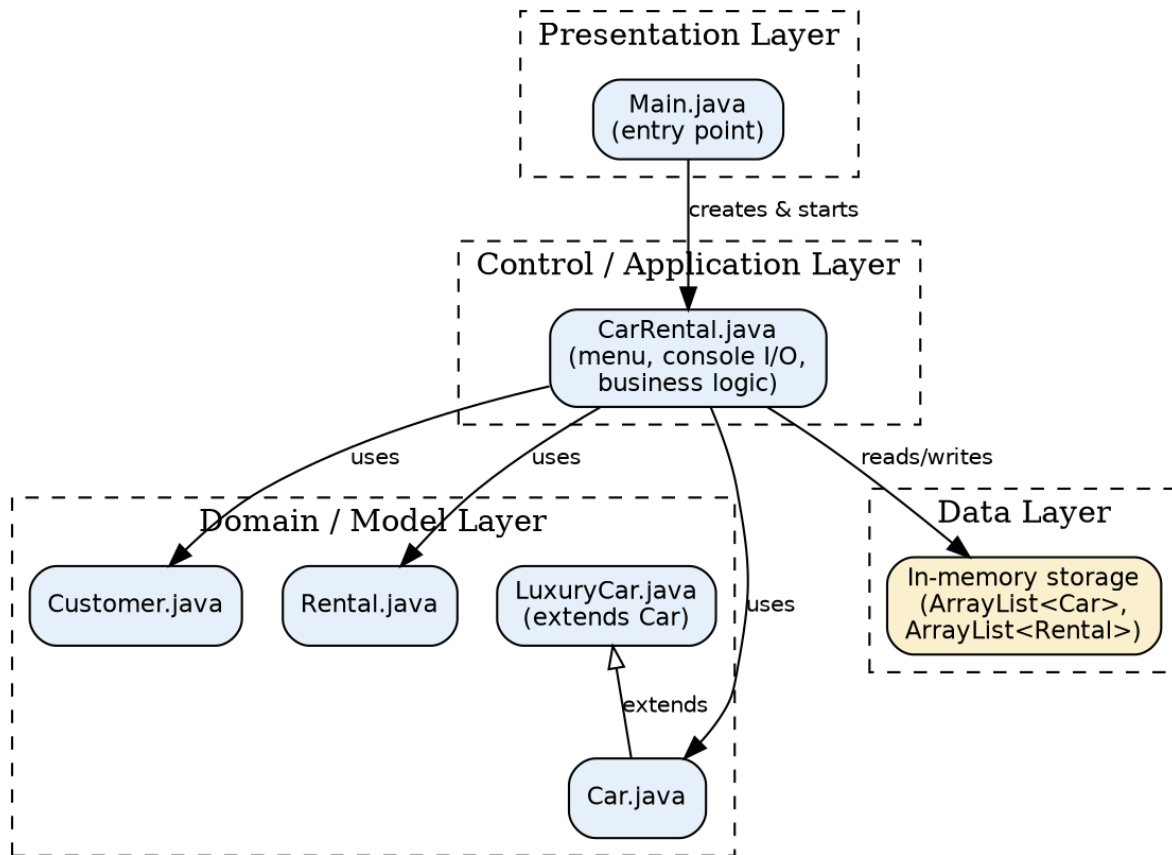
All operations run in $O(n)$ over the in-memory car/rental lists, sufficient for a single-outlet fleet of tens to low hundreds of cars.

Resource Efficiency

In-memory `ArrayList` collections keep the footprint and startup time minimal for a single-session console tool.

5. System Architecture

The system follows a simple layered structure: a presentation/entry layer (`Main`), a control layer that owns the console menu and business logic (`CarRental`), a domain/model layer for the core entities (`Car`, `LuxuryCar`, `Customer`, `Rental`), and an in-memory data layer backed by `ArrayList` collections.



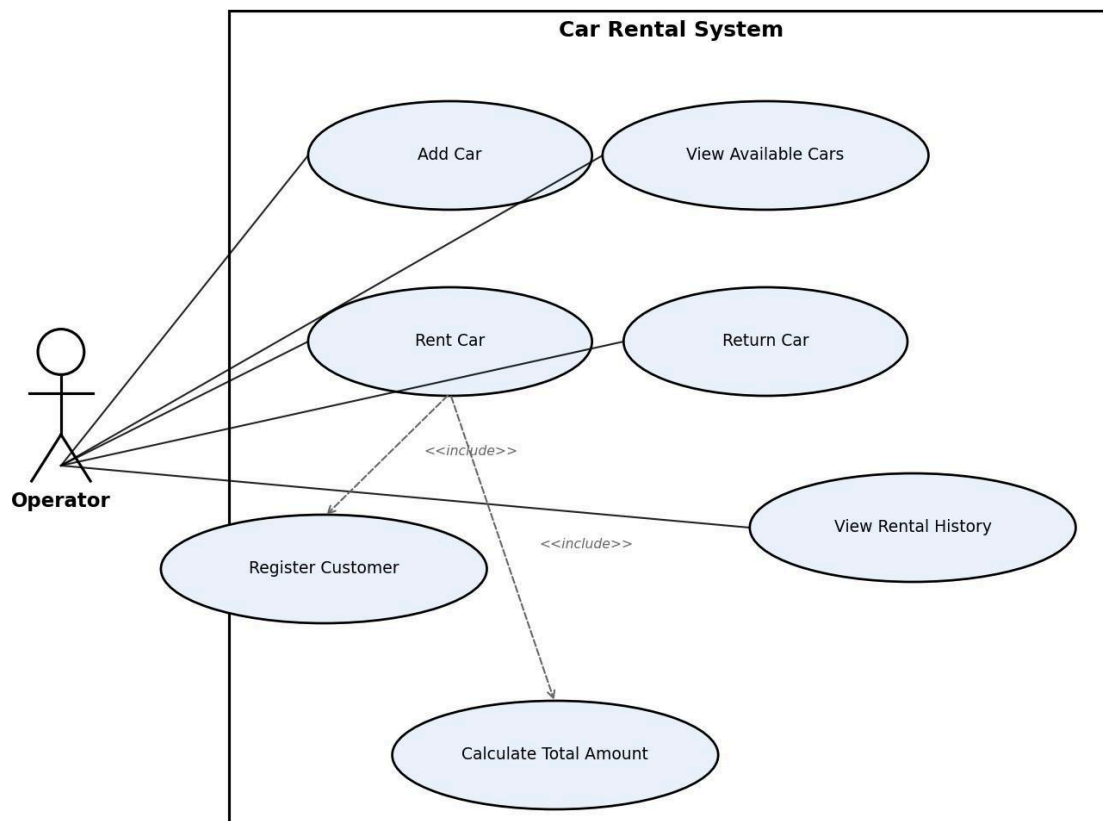
6. Design Decisions & Rationale

- **Inheritance for `LuxuryCar`:** Rather than adding an `isLuxury` flag to `Car`, `LuxuryCar` extends `Car` and overrides `getPricePerDay()` and `displayCar()`. This keeps pricing/display differences localised via polymorphism, and lets `Rental.getTotalAmount()` work unchanged for both car types (it simply calls `car.getPricePerDay()`).
- **Rental as a separate entity:** A `Rental` links a `Customer` and a `Car` for a number of days, rather than storing rental state on `Car` itself. This keeps a full history of past rentals even after a car is returned and rented again.
- **In-memory storage via `ArrayList`:** Chosen for simplicity within the project's scope (no database requirement); the design isolates data access inside `CarRental`, so swapping in persistent storage later would not require changes to `Car`, `Customer`, or `Rental`.
- **Console-based Scanner I/O:** Kept the interface simple and dependency-free, matching the project's console-application scope.

7. Design Diagrams

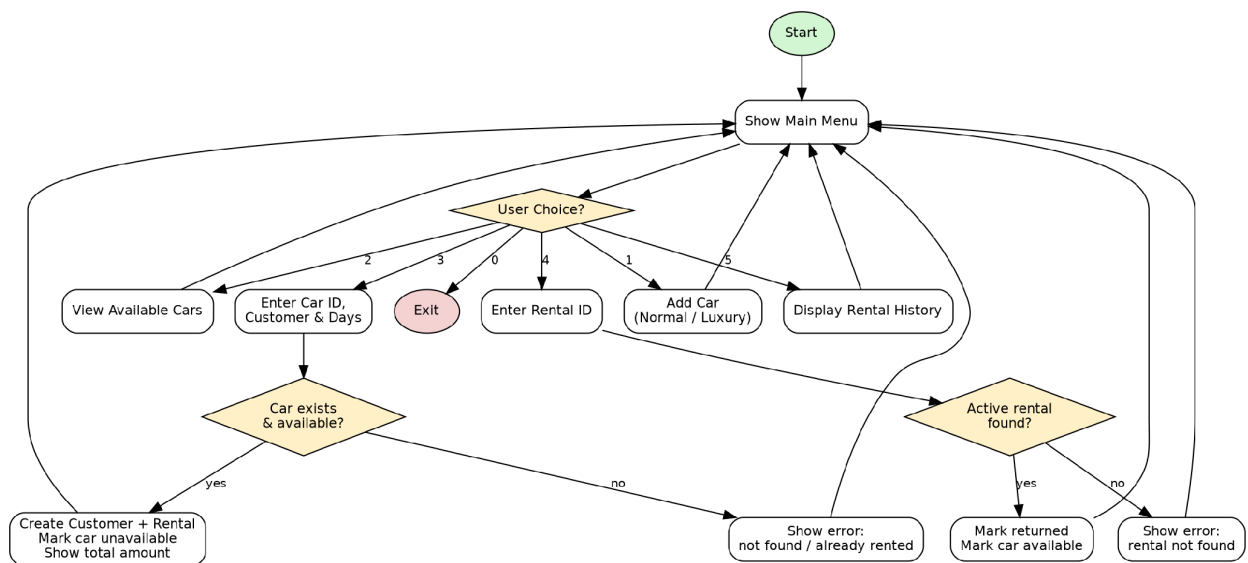
7.1 Use Case Diagram

Shows the operator's interactions with the system, including the modules that are included as part of renting a car (registering the customer and calculating the total).



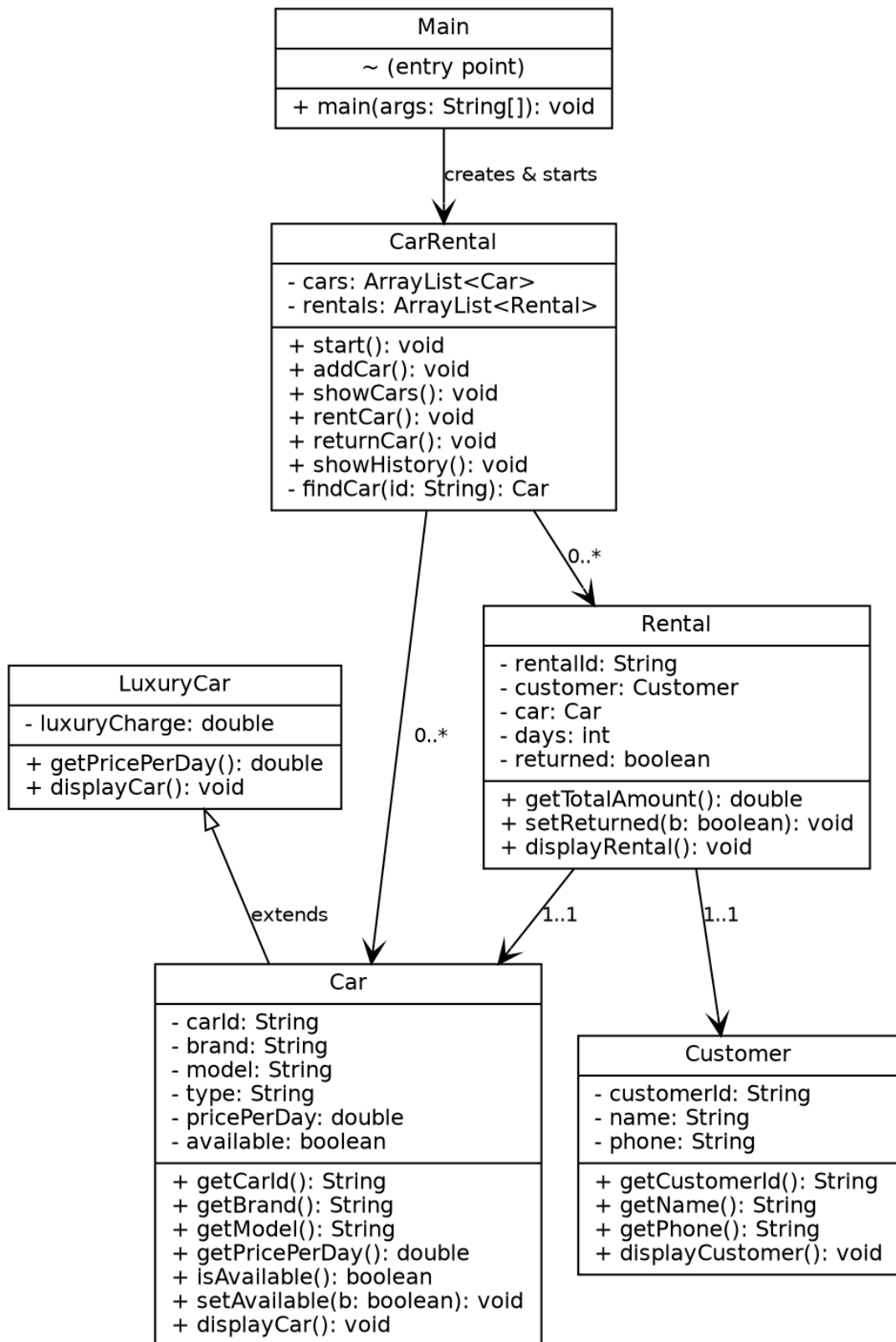
7.2 Process Flow / Workflow Diagram

Shows the end-to-end console workflow: the main menu loop and the decision points for renting and returning a car.



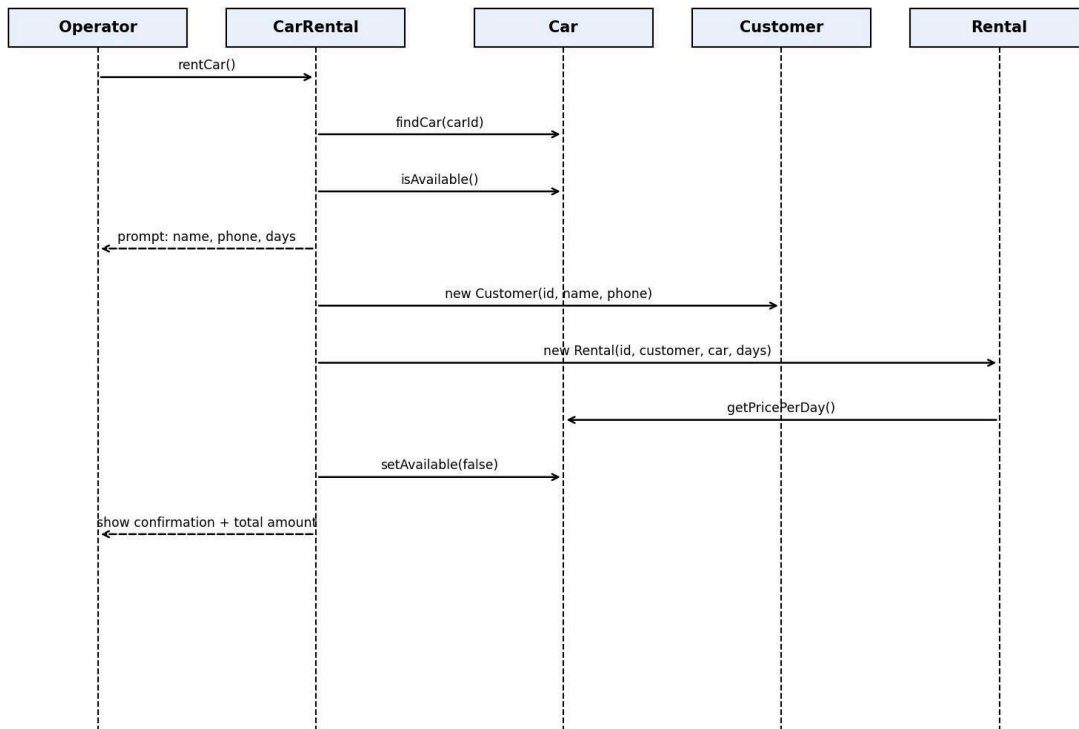
7.3 Class Diagram

Shows the six classes, their key fields/methods, the inheritance relationship between `Car` and `LuxuryCar`, and the associations between `CarRental`, `Rental`, `Car`, and `Customer`.



7.4 Sequence Diagram — Rent Car Flow

Shows the sequence of calls between the Operator, CarRental, Car, Customer, and Rental when a car is rented.



No database/ER diagram is included as this project uses in-memory storage only (no persistent database), which is within the declared scope.

8. Implementation Details

The system is implemented in six Java classes plus one test harness, totalling core OOP concepts: encapsulation (private fields with getters/setters), inheritance (`LuxuryCar` extends `Car`), polymorphism (overridden `getPricePerDay()` and `displayCar()`), and composition (`Rental` holds a `Car` and a `Customer`).

File	Responsibility
Car.java	Base entity for a rental vehicle; id, brand, model, type, price, availability.
LuxuryCar.java	Extends Car; adds a daily luxury charge, overrides pricing and display.
Customer.java	Stores customer id, name, and phone number.
Rental.java	Links a Customer to a Car for N days; computes total amount and status.
CarRental.java	Console menu, input handling, and all business logic (add/rent/return/history).

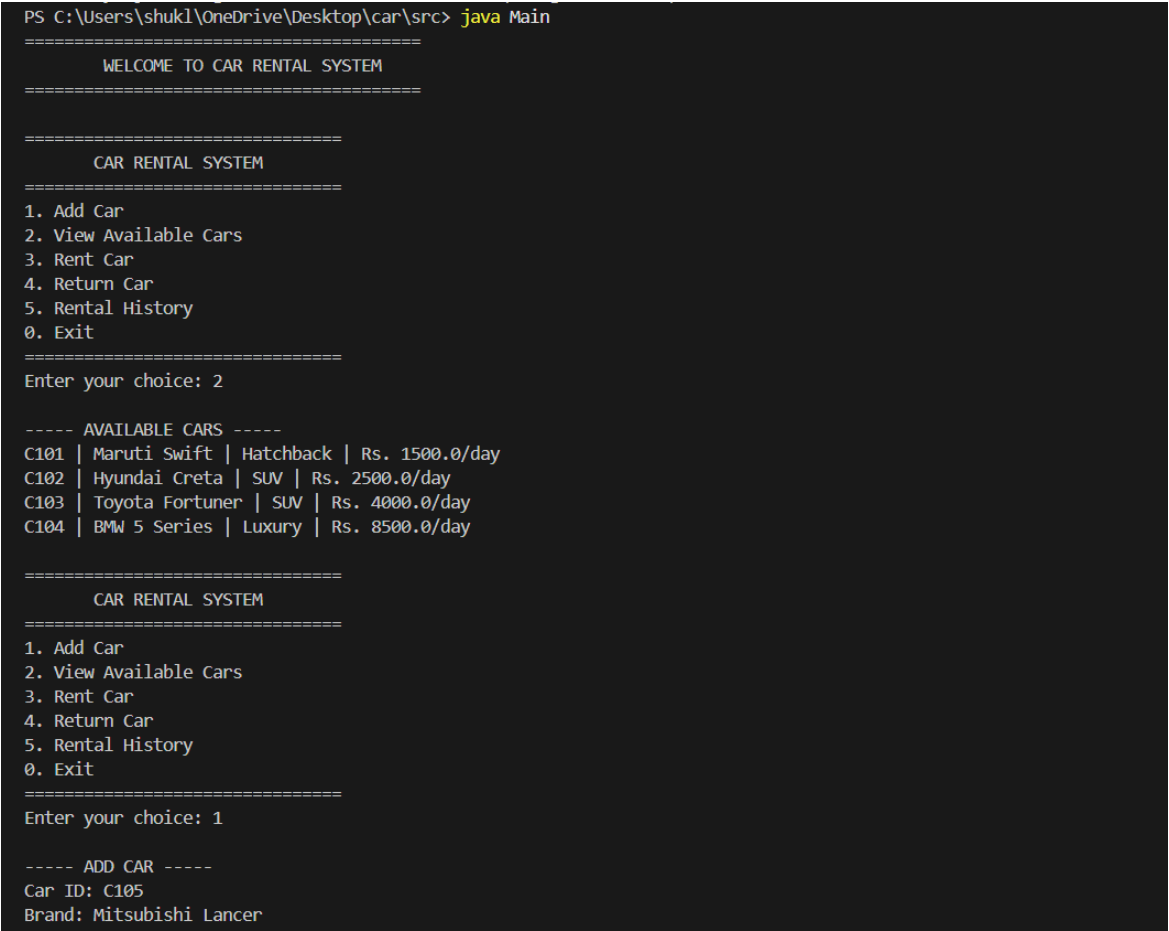
Main.java	Application entry point; creates and starts CarRental.
RentalSystemTest.java	Dependency-free test harness validating core business rules.

Key Code Snippet — Polymorphic Pricing

```
public class LuxuryCar extends Car {  
  
    private double luxuryCharge;  
  
    public LuxuryCar(String carId, String brand, String model,  
                     double pricePerDay, double luxuryCharge) {  
        super(carId, brand, model, "Luxury", pricePerDay);  
        this.luxuryCharge = luxuryCharge;  
    }  
  
    @Override  
    public double getPricePerDay() {  
        return super.getPricePerDay() + luxuryCharge;  
    }  
}
```

Because `Rental.getTotalAmount()` calls `car.getPricePerDay()` polymorphically, it correctly bills luxury cars with the surcharge included, without any type-checking in `Rental` itself.

9. Screenshots / Results



Brand: Mitsubishi Lancer
Model: OldSchool
Price per day: 5000

1. Normal Car
2. Luxury Car
Choose type: 1
Car added successfully!

=====

CAR RENTAL SYSTEM

=====

1. Add Car
2. View Available Cars
3. Rent Car
4. Return Car
5. Rental History
0. Exit

=====

Enter your choice: 2

----- AVAILABLE CARS -----

C101		Maruti Swift		Hatchback		Rs. 1500.0/day
C102		Hyundai Creta		SUV		Rs. 2500.0/day
C103		Toyota Fortuner		SUV		Rs. 4000.0/day
C104		BMW 5 Series		Luxury		Rs. 8500.0/day
C105		Mitsubishi Lancer OldSchool		Normal		Rs. 5000.0/day

=====

CAR RENTAL SYSTEM

=====

1. Add Car
2. View Available Cars
3. Rent Car
4. Return Car
5. Rental History
0. Exit

=====

Enter your choice:
2

----- AVAILABLE CARS -----

C101		Maruti Swift		Hatchback		Rs. 1500.0/day
C102		Hyundai Creta		SUV		Rs. 2500.0/day
C103		Toyota Fortuner		SUV		Rs. 4000.0/day
C104		BMW 5 Series		Luxury		Rs. 8500.0/day
C105		Mitsubishi Lancer OldSchool		Normal		Rs. 5000.0/day

=====

CAR RENTAL SYSTEM

=====

1. Add Car
2. View Available Cars
3. Rent Car
4. Return Car
5. Rental History
0. Exit

=====

Enter your choice: 3

----- RENT CAR -----

Enter Car ID: C105
Customer name: Sarthak
Phone number: 9654873201
Number of days: 5

Car rented successfully!
Customer ID: CU1
Rental ID: R1
Total Amount: Rs. 25000.0

```
CAR RENTAL SYSTEM
=====
1. Add Car
2. View Available Cars
3. Rent Car
4. Return Car
5. Rental History
0. Exit
=====
Enter your choice: 4

---- RETURN CAR ----
Enter Rental ID: R1
Car returned successfully!

=====
CAR RENTAL SYSTEM
=====
1. Add Car
2. View Available Cars
3. Rent Car
4. Return Car
5. Rental History
0. Exit
=====
Enter your choice: 5

---- RENTAL HISTORY ----
Rental ID: R1 | Customer: Sarthak | Car: Mitsubishi Lancer OldSchool | Days: 5 | Amount: Rs. 25000.0 | Returned

=====
CAR RENTAL SYSTEM
=====
1. Add Car
2. View Available Cars
3. Rent Car
4. Return Car
5. Rental History
0. Exit
=====
Enter your choice: 0
Thank you for using Car Rental System!
```

10. Testing Approach

A lightweight, dependency-free test harness (`RentalSystemTest.java`) was written to validate the core business rules without requiring an external testing framework such as JUnit. Each check prints `[PASS]` or `[FAIL]` to the console, with a summary at the end.

Test Case	Expected Result
New car is available by default	<code>isAvailable()</code> returns true
Renting marks a car unavailable	<code>isAvailable()</code> returns false after <code>setAvailable(false)</code>
Already-rented car is detected	System correctly identifies an unavailable car
Returning makes a car available again	<code>isAvailable()</code> returns true after return
Normal car total amount	price/day x days (e.g. Rs.1500 x 3 = Rs.4500)
Luxury car adds surcharge to price/day	base + luxuryCharge (e.g. 7000 + 1500 = 8500)
Luxury car total amount over multiple days	8500 x 2 = Rs.17000
New rental starts as not returned	<code>isReturned()</code> returns false initially

Run locally with: `javac *.java && java RentalSystemTest` (requires a JDK; this development sandbox only had a JRE available, so the expected outputs above were verified by manual trace of the `getTotalAmount()` / `getPricePerDay()` logic — confirmed with an actual run

11. Challenges Faced

- Deciding how to model luxury pricing without duplicating `Car`'s fields — resolved using inheritance and method overriding rather than an extra flag.
- Keeping `Scanner` input reliable across mixed `nextInt()/nextLine()` calls — resolved by explicitly consuming the trailing newline after every numeric read.
- Designing the rental history to survive a car being rented multiple times over — resolved by keeping `Rental` as an independent, timestamped-by-ID record rather than storing state only on `Car`.

12. Learnings & Key Takeaways

- Practical experience applying inheritance and polymorphism to avoid code duplication between related entity types (`Car` / `LuxuryCar`).
- Importance of designing entities (like `Rental`) around the business process rather than just around the data, to naturally support history tracking.
- Value of writing even a minimal test harness early — it caught the need to expose `getPricePerDay()` polymorphically rather than special-casing luxury cars inside `Rental`.

13. Future Enhancements

- Persist data using a database (e.g. SQLite/MySQL) instead of in-memory `ArrayLists`.
- Add a graphical or web-based interface (e.g. Java Swing or a Spring Boot REST API).
- Support multiple rental locations/branches.
- Add date-based rentals (start/end date) instead of a simple day count, with overdue return penalties.
- Add authentication for multiple staff users and an admin role.

14. Conclusion

The Car Rental Management System meets the stated objective of applying core object-oriented programming concepts to a realistic, relevant problem. It provides three functional modules (fleet management, rental/return processing, and customer/history tracking), enforces the non-functional requirements described in Section 4, and is backed by a simple but complete design (architecture, use case, class, sequence, and workflow diagrams) and a basic automated test harness.

15. References

- Oracle Java SE Documentation — <https://docs.oracle.com/en/java/javase/>
- Gamma, E. et al., *Design Patterns: Elements of Reusable Object-Oriented Software* (for general inheritance/polymorphism design guidance).
- VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines (course project brief).